



Assignment 03 Report

CE327: General Purpose Computing with GPU

Student Name: Rayyan Hassan Salman

Reg Number: 2023601

Faculty of Computer Sciences and Engineering
Ghulam Ishaq Institute of Engineering Sciences and Technology

Objective: Compile and run serial image filters, design equivalent CUDA GPU kernels, and apply optimization techniques (especially shared-memory tiling) to improve performance and compare CPU vs GPU timing.

March 31, 2026

Edge Detection: Serial to Parallel Acceleration

Task Summary

The serial edge detection filter computes Sobel gradients on a grayscale image and thresholds the gradient magnitude to classify each pixel as edge/non-edge. The GPU version maps one thread per output pixel and preserves the same filter logic.

Implementation Steps

1. Verified serial implementation output and baseline runtime.
2. Wrote a dedicated CUDA kernel (`edge_detect_kernel`) for Sobel computation.
3. Added a separate host launcher function (`edge_detect`) for memory allocation, transfers, kernel launch, and copy-back.
4. Used shared-memory tiled processing with halo and corner loading.
5. Added halo loading and corner handling to correctly process tile boundaries.
6. Measured runtime using CUDA events and converted all timing to milliseconds.

Important Host Wrapper Snippet

Listing 1: Separate kernel-calling function used in edge parallel code

```

1 void edge_detect(const PGM_image* in, PGM_image* edges, int threshold
2 )
3 {
4     int width = in->width, height = in->height;
5     size_t nbytes = (size_t)width * height * sizeof(unsigned char);
6     unsigned char *d_in = NULL, *d_out = NULL;
7
8     cudaSafeCall(cudaMalloc(&d_in, nbytes));
9     cudaSafeCall(cudaMalloc(&d_out, nbytes));
10    cudaSafeCall(cudaMemcpy(d_in, in->bmap, nbytes,
11                          cudaMemcpyHostToDevice));
12
13    dim3 block(block_dim, block_dim);
14    dim3 grid((width + block_dim - 1) / block_dim,
15             (height + block_dim - 1) / block_dim);
16
17    edge_detect_kernel<<<grid, block>>>(d_in, d_out, width, height,
18                                     threshold);
19    cudaCheckError();
20
21    cudaSafeCall(cudaMemcpy(edges->bmap, d_out, nbytes,
22                          cudaMemcpyDeviceToHost));
23    cudaSafeCall(cudaFree(d_in));
24    cudaSafeCall(cudaFree(d_out));
25 }

```

Performance (Standardized to ms)

Version	Reported Time	Standardized Time (ms)
Serial Edge Detection	16.000 ms	16.000 ms
Parallel Edge Detection	0.293 ms	0.293 ms

Speedup:

$$extSpeedup = \frac{16.000}{0.293} \approx 54.61 \times$$

Denoising (Median-of-Five) and Overall Findings

Denoising Parallelization Approach

The denoising filter applies a median-of-five stencil per channel (R, G, B). The GPU implementation follows the same structure used in edge detection:

1. A dedicated CUDA kernel computes one output pixel per thread.
2. A separate host function handles device memory, transfers, and launch configuration.
3. The launch geometry uses 2D blocks and grids derived from image dimensions (`block_dim = 32`).
4. Timing uses CUDA events to accurately measure GPU-side processing.

Key Optimization Ideas from Chapter 6 Applied

1. **Thread-level parallelism:** one thread per pixel for high concurrency.
2. **Memory hierarchy usage:** shared-memory tiling in edge detection, and shared-memory neighborhood loading in denoising.
3. **Coalesced access tendency:** row-major mapping improves contiguous memory reads/writes.
4. **Reduced overhead structure:** separate kernel and launcher functions improve maintainability and profiling clarity.

Important Note on Extra Work in `edge.cu`

The richer work in `edge.cu` was mainly the shared-memory tiled Sobel path, and your edge parallel file now reflects that:

1. Explicit halo and corner loads.
2. Synchronization with `__syncthreads()` before stencil math.
3. Separate kernel and kernel-calling host function.

That same structure should be retained in the parallel submission for consistency and performance.

Conclusion

Both filters were first validated in serial form and then moved to CUDA with equivalent output behavior. The edge detection case shows strong acceleration after GPU parallelization and additional gains from tiled shared memory. This assignment demonstrates that stencil-like image operators benefit significantly from GPU execution when memory access and boundary cases are handled carefully.

Denoising Performance (Standardized to ms)

oprule Version	Reported Time	Standardized Time (ms)
Serial Denoising	48.000 ms	48.000 ms
Parallel Denoising	0.988 ms	0.988 ms

Speedup:

$$extSpeedup = \frac{48.000}{0.988} \approx 48.58 \times$$

End of Report